

Application Python

Contexte

Vous venez d'intégrer une entreprise spécialisée dans l'Intelligence Artificielle.

Les Data Scientists manipulent quotidiennement des centaines de datasets (CSV et JSON). Avant de commencer les traitements avec Pandas, ils souhaitent disposer d'une application Python permettant de gérer ces jeux de données.

Votre mission consiste à développer progressivement une application console permettant de :

- gérer un catalogue de jeux de données ;
- enregistrer leurs caractéristiques ;
- effectuer des recherches ;
- afficher des statistiques ;
- sauvegarder les informations dans des fichiers ; ☐ charger automatiquement les données enregistrées ; ☐ gérer les erreurs d'utilisation.

- 1) Créez datasetManager, le dossier racine de l'application
- 2) Créez le fichier main.py, point d'entrée de l'application, pour contenir tout le code de la partie 1 à la partie 8.

Partie 1 : Types de base, variables, Entrées et sorties

- 3) Demandez à l'utilisateur de saisir les métadonnées d'un dataset :
 - nom du dataset
 - domaine
 - nombre de lignes
 - nombre de colonnes
 - taille en Mo
 - format (csv ou json)
 - public (true ou false)

```
print("Veuillez saisir les métadonnées du dataset :")
nom_dataset = input("Nom du dataset : ")
domaine = input("Domaine : ")
nombre_lignes = int(input("Nombre de lignes : "))
nombre_colonnes = int(input("Nombre de colonnes : "))
taille_mo = float(input("Taille en Mo : "))
format_dataset = input("Format (csv ou json) : ")
public = input("Public (true ou false) : ").lower() == "true"
```

```
Veillez saisir les métadonnées du dataset :  
Nom du dataset : IA  
Domaine( Santé, Finance, Agriculture, Transport, Education) : Education  
Nombre de lignes : 89  
Nombre de colonnes : 76  
Taille en Mo : 78  
Format (csv ou json) : CSV  
Public (true ou false) : TRUE
```

4) Affichez ensuite un résumé formaté.

```
print("\nRésumé du dataset :")  
print(f"Nom : {nom_dataset}")  
print(f"Domaine : {domaine}")  
print(f"Nombre de lignes : {nombre_lignes}")  
print(f"Nombre de colonnes : {nombre_colonnes}")  
print(f"Taille : {taille_mo} Mo")  
print(f"Format : {format_dataset}")  
print(f"Public : {public}")
```

```
Résumé du dataset :  
Nom : IA  
Domaine : Education  
Nombre de lignes : 89  
Nombre de colonnes : 76  
Taille : 78.0 Mo  
Format : CSV  
Public : True  
=====
```

Partie 2 – Structures de contrôle

5) Créez un menu interactif (provisoire)

```
=====
```

1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher

4. Quitter

=====

Le programme reste actif jusqu'à ce que l'utilisateur choisisse Quitter.

```
print("=====")
print("\nMenu interactif :")
while True:
    print("\n1. Ajouter un dataset")
    print("2. Afficher les datasets")
    print("3. Rechercher")
    print("4. Quitter")

    choix = input("Veuillez choisir une option (1-4) : ")

    if choix == "1":
        print("Option 1 sélectionnée : Ajouter un dataset")
    elif choix == "2":
        print("Option 2 sélectionnée : Afficher les datasets")
    elif choix == "3":
        print("Option 3 sélectionnée : Rechercher")
    elif choix == "4":
        print("Option 4 sélectionnée : Quitter")
        break

print("=====")
```

```
Menu interactif :

1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
Veuillez choisir une option (1-4) : 1
Option 1 sélectionnée : Ajouter un dataset
```

Partie 3 – Dictionnaires

6) Créez un dictionnaire pour stocker les métadonnées de chaque dataset

Exemple :

```
{  
  
"nom": "Titanic",  
"domaine": "Transport",  
"lignes": 891,  
"colonnes": 12,  
"taille": 48,  
"format": "CSV",  
"public": True  
}
```

```
Dataset = {  
    "nom": nom_dataset,  
    "domaine": domaine,  
    "lignes": nombre_lignes,  
    "colonnes": nombre_colonnes,  
    "taille": taille_mo,  
    "format": format_dataset,  
    "public": public  
}
```

Partie 4 – Tuples

7) Créez un tuple contenant les domaines autorisés.

("Santé", "Finance", "Agriculture", "Transport", "Education")

```
domaines_autorises = (
    "Santé",
    "Finance",
    "Agriculture",
    "Transport",
    "Education"
)
```

8) Vérifiez que le domaine saisi, à la question 3, appartient au tuple.

```
domaine = input("Domaine( Santé, Finance, Agriculture, Transport, Education) : ")
while domaine not in domaines_autorises:
    print("Domaine invalide. Veuillez choisir parmi les domaines autorisés :", domaines_autorises)
    domaine = input("Domaine : ")
```

```

Veuillez saisir les métadonnées du dataset :
Nom du dataset : dataset_test
Domaine( Santé, Finance, Agriculture, Transport, Education) : test
Domaine invalide. Veuillez choisir parmi les domaines autorisés : ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education')
Domaine : auteur
Domaine invalide. Veuillez choisir parmi les domaines autorisés : ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education')
Domaine : sante
Domaine invalide. Veuillez choisir parmi les domaines autorisés : ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education')
Domaine : santé
Domaine invalide. Veuillez choisir parmi les domaines autorisés : ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education')
Domaine : Santé
Nombre de lignes : 
```

Partie 5 – Listes

9) Créez une liste contenant les datasets. Chaque ajout est enregistré dans la liste

```
liste_datasets = []
```

10) Ajoutez les fonctionnalités :

- ajouter
- trier
- rechercher
- modifier

- supprimer

```
print("\n===== MENU DATASETS =====")
print("1. Ajouter un dataset")
print("2. Afficher les datasets")
print("3. Rechercher un dataset")
print("4. Modifier un dataset")
print("5. Supprimer un dataset")
print("6. Trier les datasets")
print("7. Statistiques")
print("8. Quitter")

choix = input("Votre choix : ")
```

```
===== MENU DATASETS =====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher un dataset
4. Modifier un dataset
5. Supprimer un dataset
6. Trier les datasets
7. Statistiques
8. Quitter
Votre choix : 1

--- Ajout d'un dataset ---
Nom : DATASET3
Domaine : education
Nombre de lignes : 678
Nombre de colonnes : 988
Taille (Mo) : 900
Format (csv/json) : JSON
Public (true/false) : FALSE
Dataset ajouté avec succès !
```

Partie 6 – Compréhensions (Listes et Dictionnaires)

11) Affichez les statistiques comme par exemple :

- Nombre de datasets : 18
- Nombre total de lignes : 2 540 000
- Nombre moyen de colonnes : 27
- Datasets publics : 8
- Datasets privés : 10
- Nombre de datasets au format CSV : 13 Nombre de datasets au format JSON : 5
- Répartition par domaine o Santé : 5 o Finance : 3 o Agriculture : 2 o
Transport : 4 o Education : 4

Testez, au fur et à mesure, avec des datasets d'exemples

```
elif choix == "7":
    print("\n==== STATISTIQUES =====")
    if len(liste_datasets)==0:
        print("Aucun dataset.")
    else:
        nombre_dataset = len(liste_datasets)
        total_lignes = 0
        total_colonnes = 0
        public = 0
        prive = 0
        csv = 0
        json = 0
        domaines = {}
        for dataset in liste_datasets:
            total_lignes += dataset["lignes"]
            total_colonnes += dataset["colonnes"]
            if dataset["public"]:
                public += 1
            else:
                prive += 1
            if dataset["format"].lower()=="csv":
                csv += 1
            elif dataset["format"].lower()=="json":
                json += 1
```

```

        csv += 1
    elif dataset["format"].lower()=="json":
        json += 1
        domaine = dataset["domaine"]
        if domaine in domaines:
            domaines[domaine] += 1
        else:
            domaines[domaine] = 1
    moyenne_colonnes = total_colonnes / nombre_dataset
    print("Nombre de datasets :", nombre_dataset)
    print("Total lignes :", total_lignes)
    print("Moyenne colonnes :", moyenne_colonnes)
    print("Publics :", public)
    print("Privés :", prive)
    print("CSV :", csv)
    print("JSON :", json)
    print("\nRépartition domaines :")
    for domaine, nombre in domaines.items():
        print(
            domaine,
            ":",
            nombre
        )

```

```

===== STATISTIQUES =====
Nombre de datasets : 1
Total lignes : 678
Moyenne colonnes : 988.0
Publics : 0
Privés : 1
CSV : 0
JSON : 1

Répartition domaines :
education : 1

```

Partie 7 – Fichiers

12) Créez le fichier datasets.csv pour :

- sauvegarder les données

- recharger et afficher les données

```
import csv

with open("datasets.csv", "w", newline="", encoding="utf-8") as fichier:
    writer = csv.writer(fichier)
    writer.writerow([
        "nom",
        "domaine",
        "lignes",
        "colonnes",
        "taille",
        "format",
        "public"
    ])
    for dataset in liste_datasets:
        writer.writerow([
            dataset["nom"],
            dataset["domaine"],
            dataset["lignes"],
            dataset["colonnes"],
            dataset["taille"],
            dataset["format"],
            dataset["public"]
        ])
print("Sauvegarde terminée.")
```

```
liste = []

with open("datasets.csv", "r", encoding="utf-8") as fichier:
    lecteur = csv.DictReader(fichier)
    for ligne in lecteur:
        dataset = {
            "nom": ligne["nom"],
            "domaine": ligne["domaine"],
            "lignes": int(ligne["lignes"]),
            "colonnes": int(ligne["colonnes"]),
            "taille": float(ligne["taille"]),
            "format": ligne["format"],
            "public": ligne["public"] == "True"
        }
        liste.append(dataset)
```

```
def afficher_datasets(liste):
    liste = recharger()
    for dataset in liste:
        print(dataset)
```

Partie 8 – Exceptions Plusieurs

erreurs sont constatées :

- l'utilisateur saisit un texte au lieu d'un nombre ;

```
def saisir_entier(message):
    """
    Demande un entier à l'utilisateur.
    La saisie est répétée tant qu'elle est invalide.
    """

    while True:
        try:
            return int(input(message))

        except ValueError:
            print("✗ Erreur : veuillez saisir un nombre entier.")
```

```

def saisir_reel(message):
    """
    Demande un nombre décimal.
    """

    while True:

        try:

            return float(input(message))

        except ValueError:

            print("✗ Erreur : veuillez saisir un nombre valide.")

```

- le fichier n'existe pas ;

```

    # Vérification d'un fichier vide

    if len(liste) == 0:

        print("⚠ Le fichier datasets.csv est vide.")

    else:

        print("✓ Chargement terminé.")

    except FileNotFoundError:

        print("✗ Le fichier datasets.csv est introuvable.")

    except Exception as erreur:

        print("✗ Une erreur est survenue :", erreur)

    return liste

```

- le dataset recherché n'existe pas ;
- le fichier est vide.

```

def rechercher_dataset(liste):
    """
    Recherche un dataset par son nom.
    """

    nom = input("Nom du dataset : ")

    for dataset in liste:

        if dataset["nom"].lower() == nom.lower():

            print("\n===== DATASET TROUVÉ =====")

            for cle, valeur in dataset.items():
                print(f"{cle.capitalize()} : {valeur}")

            return

    print("❌ Aucun dataset ne porte ce nom.")

```

13) Gérez ces exceptions pour que le programme continue de fonctionner.

Partie 9 – Fonctions

14) Refactorisez le programme en créant les fonctions suivantes :

```

afficher_menu() ajouter_dataset()
afficher_datasets()
supprimer_dataset()
rechercher_dataset() trier_dataset()
modifier_dataset() sauvegarder()
recharger()
statistiques()

```

```
while True:
    afficher_menu()
    choix = input("Votre choix : ")

    if choix == "1":
        ajouter_dataset(liste_datasets)

    elif choix == "2":
        afficher_datasets(liste_datasets)

    elif choix == "3":
        rechercher_dataset(liste_datasets)

    elif choix == "4":
        modifier_dataset(liste_datasets)

    elif choix == "5":
        supprimer_dataset(liste_datasets)
```

```
        trier_dataset(liste_datasets)

    elif choix == "7":

        sauvegarder(liste_datasets)

    elif choix == "8":

        liste_datasets = recharger()

    elif choix == "9":

        statistiques(liste_datasets)

    elif choix == "10":

        print("Fin du programme.")

    break
```

Partie 10 – Modules

Jusqu'à présent, toute l'application est contenue dans un seul fichier.

15) Pour éviter des problèmes de maintenance, répartissez le code dans plusieurs modules.

```
main.py menu.py
gestion.py
statistiques.py
```

```
Répertoire : C:\Users\rokha\OneDrive\Documents\ODC\Python\datasetManager

Mode                LastWriteTime         Length Name
----                -
-a----            8/5/2026   2:42 PM             88 datasets.csv
-a----            8/5/2026   7:33 PM          7837 gestion.py
-a----            8/5/2026   4:27 PM          1092 main.py
-a----            8/5/2026   3:56 PM           510 menu.py
-a----            8/5/2026   3:42 PM           18 README.md
-a----            8/5/2026   4:00 PM          1660 statistiques.py

PS C:\Users\rokha\OneDrive\Documents\ODC\Python\datasetManager> |
```

Partie 11 – Packages

16) Découpez l'application en packages comme suit :

datasetManager/ |—

main.py

|— datasets/

| __init__.py

| ges on.py

| sta s ques.py

|— interface/

| __init__.py

| menu.py

| affichage.py

|— stockage/

| __init__.py

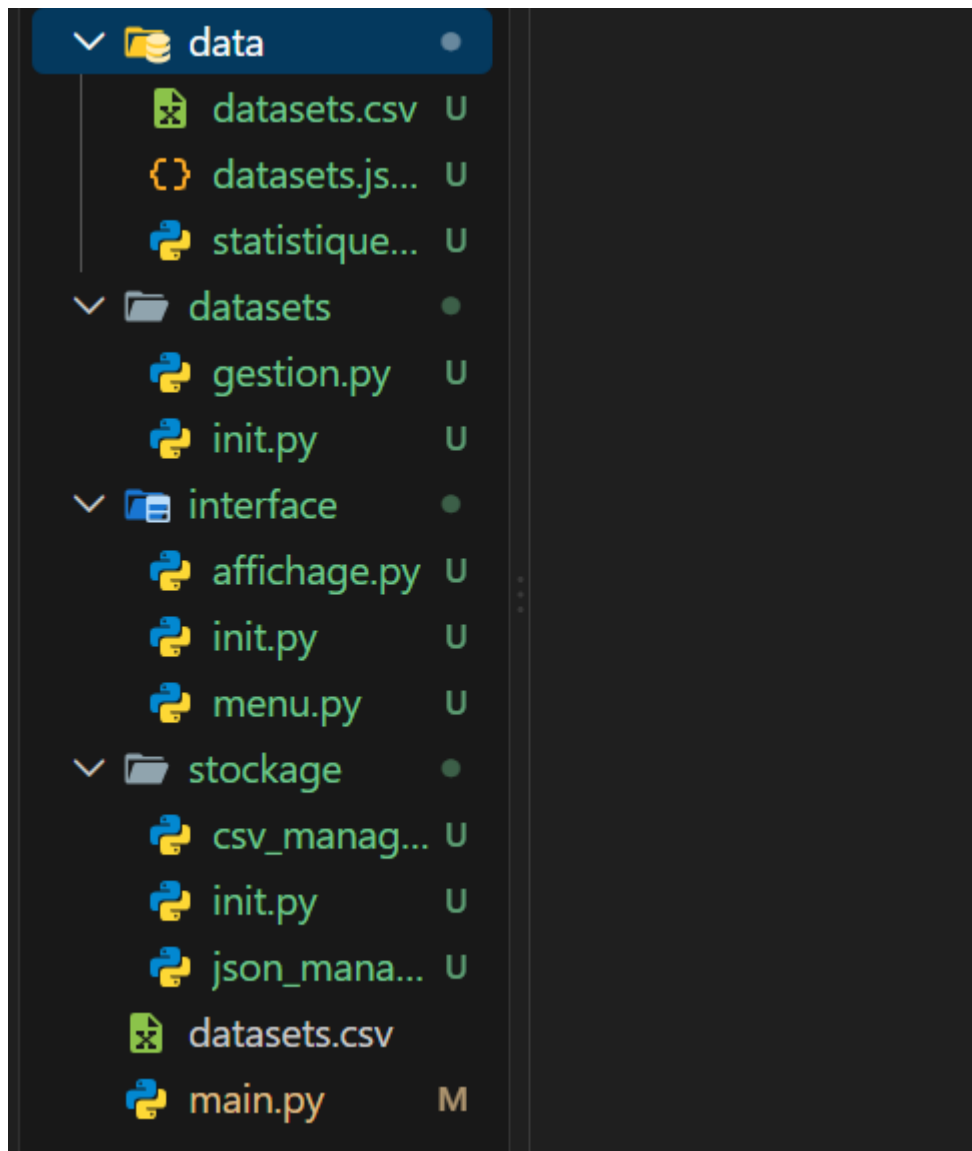
| csv_manager.py

| json_manager.py

|— data/

| datasets.csv

| datasets.json



Partie 12 : Bonus

Proposez et réalisez toute fonctionnalité pertinente et utile qu'on pourrait rajouter à l'application.

```

# =====
#                                     Bonus
# =====

def generer_rapport_txt(liste_datasets):
    """Génère un rapport d'analyse textuel complet des datasets."""
    if not liste_datasets:
        print("/!\ Aucun dataset en mémoire pour générer un rapport.")
        return

    # Calculs des statistiques
    total_datasets = len(liste_datasets)
    total_lignes = sum(d.get('lignes', 0) for d in liste_datasets)
    total_taille = sum(d.get('taille', 0) for d in liste_datasets)

    # Compte des formats
    nb_csv = sum(1 for d in liste_datasets if d.get('format', '').lower() == 'csv')
    nb_json = sum(1 for d in liste_datasets if d.get('format', '').lower() == 'json')

    # Création du dossier data s'il n'existe pas
    os.makedirs("data", exist_ok=True)
    chemin_rapport = "data/rapport_statistiques.txt"

```

```

try:
    with open(chemin_rapport, "w", encoding="utf-8") as fichier:
        fichier.write("=====\n")
        fichier.write("          RAPPORT AUTOMATIQUE DES DATASETS          \n")
        fichier.write(f"   Généré le : {datetime.now().strftime('%d/%m/%Y à %H:%M:%S')} \n")
        fichier.write("=====\n\n")

        fichier.write("--- STATISTIQUES GLOBALES ---\n")
        fichier.write(f"Nombre total de datasets : {total_datasets}\n")
        fichier.write(f"Nombre cumulé de lignes   : {total_lignes:,}\n".replace(',', ','))
        fichier.write(f"Volume total stocké      : {total_taille:.2f} Mo\n\n")

        fichier.write("--- RÉPARTITION PAR FORMAT ---\n")
        fichier.write(f"Fichiers CSV   : {nb_csv}\n")
        fichier.write(f"Fichiers JSON  : {nb_json}\n\n")

        fichier.write("--- INVENTAIRE DES JEUX DE DONNÉES ---\n")
        fichier.write(f"{'Nom':<15} | {'Domaine':<15} | {'Format':<8} | {'Taille':<15}\n")
        fichier.write("-" * 56 + "\n")
        for d in liste_datasets:
            fichier.write(f"{d.get('nom', 'Inconnu'):<15} | {d.get('domaine', 'N/A')<15} | {d.get('format', 'N/A')<8} | {d.get('taille', 0):<15}\n")

        fichier.write("\n=====\n")

```

```

        fichier.write(f" {u.get( 'nom' , 'Inconnu' )}<15} | {u.get( 'domaine' , 'N/A' )}<15} \n")

        fichier.write("\n===== \n")
        fichier.write("Fin du rapport.\n")

    print(f" Fin du rapport générée avec succès dans : {chemin_rapport}")

except Exception as e:
    print(f"!\\ Erreur lors de la génération du rapport : {e}")

```

Livrables attendus

À la fin de l'application, chaque apprenant devra fournir :

1. le dossier datasetManager racine de l'application.
2. un document pdf présentant la réalisation de l'application, illustrée par la réponse en dessous de chaque question

Modalité de livraison

Chaque apprenant pousse les livrables sur un dépôt public de son compte Github personnel

Quelques conseils

Rédigez le livrable pdf au fur et à mesure du traitement des questions.

Poussez les livrables sur Github le plus tôt possible et par tâche.